

Servicio de Enseñanza Nacional de Aprendizaje SENA
Facultad de Ingeniería de Software

Taller AA2-EV01 – Creación de caso estudio y consumo de APIs RESTful con Node.js – Dependencias

Juan Diego Castañeda González

3606697. Desarrollo Backend con Node.JS y MongoDB

Docente. Olga Maritza Narvaez Sánchez

3 de octubre de 2026

Índice general

1 Resumen **2**

2 Introducción **3**
2.1 Objetivo General 3
2.2 Objetivos Específicos 3

3 Procedimientos de Implementación **4**
3.1 Diseño del Esquema de Base de Datos 4
3.2 Configuración del Servidor y Middlewares 5
3.3 Modelado de la Base de Datos NoSQL 6
3.4 Controladores y Rutas 7
3.5 Pruebas y Evidencia de Ejecución (Comandos MongoDB Atlas) 14

4 Conclusiones **22**

1. RESUMEN

Este documento técnico se presenta el diseño, desarrollo e implementación de una API RESTful desarrolladad para la gestión de inventario de la tienda de tecnología Revival Tech. Al haber tenido la necesidad de almacenar información de tecnológica con atributos altamente variables, se optó por una arquitectura de base de datos **NoSQL** usando `MongoDB`. El backend fue construido con el entorno `Node.js` empleando el framework `Express` para el enrutamiento. Para optimizar el rendimiento y la seguridad del sistema, se integraron dependencias clave: `Mongoose` para el modelado de datos y validación de esquemas; `dotenv` para la protección de credenciales de entorno; `morgan` para el monitoreo y registro (logging) de las peticiones HTTP; y `body-parser` para la correcta extracción y transformación de los datos entrantes en las solicitudes. Se detallan a continuación los objetivos, procedimientos y la evidencia de ejecución del sistema.

2. INTRODUCCIÓN

En el ámbito de la comercialización de equipos tecnológicos, el control de inventario representa un desafío a nivel de almacenamiento de datos. Un catálogo tecnológico posee artículos con especificaciones divergentes; por ejemplo, un portátil requiere campos de procesador y RAM, mientras que un periférico requiere longitud de cable y tipo de conexión. Un esquema relacional tradicional resulta ineficiente ante esta variabilidad.

Para resolver esta problemática, el presente proyecto documenta la creación de un sistema backend basado en `Node.js` y `MongoDB`. La flexibilidad de los documentos BSON, combinada con herramientas de middleware modernas, permite construir una solución escalable y segura.

2.1. Objetivo General

Desarrollar e implementar una API RESTful escalable y modular utilizando Node.js y MongoDB para la gestión dinámica del inventario tecnológico de Revival Tech, permitiendo el almacenamiento de atributos variables mediante un modelo NoSQL orientado a documentos.

2.2. Objetivos Específicos

- **Modelar los datos:** Definir esquemas documentales flexibles e implementar la conexión a la base de datos utilizando `Mongoose`.
- **Procesar solicitudes HTTP:** Configurar el servidor y las rutas del sistema utilizando `Express` y `body-parser` para extraer e interpretar los objetos JSON enviados por los clientes.
- **Monitorear la actividad:** Implementar `morgan` como middleware para llevar un registro detallado en consola de las peticiones entrantes, facilitando el proceso de depuración.
- **Proteger la información sensible:** Asegurar las credenciales de conexión al clúster y puertos del servidor aislando dichas variables a través de `dotenv`.
- **Ejecutar operaciones CRUD:** Codificar controladores funcionales para la creación, lectura, actualización y eliminación de categorías y productos del inventario.

3. PROCEDIMIENTOS DE IMPLEMENTACIÓN

La arquitectura de la API se estructuró separando responsabilidades lógicas (Controladores, Modelos y Rutas). A continuación, se detalla la integración de las tecnologías mencionadas y los espacios para su respectiva evidencia en código.

3.1. Diseño del Esquema de Base de Datos

A continuación se presenta el modelo orientado a documentos diseñado para el sistema de inventario, donde se evidencia la relación por referencia entre los productos y sus respectivas categorías, así como la estructura flexible para las especificaciones técnicas:

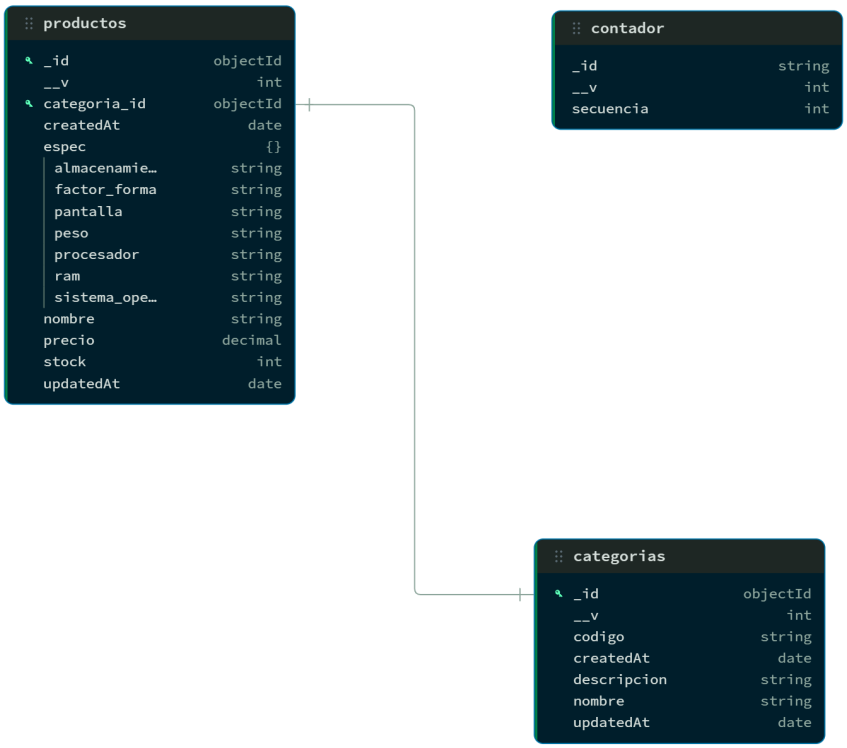


Figura 3.1: Modelo de base de datos para el inventario de TechStore (Productos, Categorías y Contador).

La parte de `Contador` sirve como un identificador que se usó como identicador de `codigo` como valor único e incremental para la colección de `Categoria` de `"CAT-###"` (ej. "CAT-001").

3.2. Configuración del Servidor y Middlewares

El punto de entrada de la aplicación gestiona la inicialización de **Express**. En esta etapa se integró **dotenv** para cargar la variable `MONGODB_URI` de forma segura. Asimismo, se inyectaron los middlewares globales: **morgan** en modo de desarrollo para registrar los verbos HTTP y tiempos de respuesta, y **body-parser** (o el parser integrado de Express) para garantizar que el `req.body` esté disponible en formato JSON estructurado.

Archivo principal de configuración del servidor (server.js)

```
// Importa  módulo para las rutas de archivos, vars entorno
   y DNS
const dns = require('node:dns');
dns.setServers(['8.8.8.8', '1.1.1.1']);
const path = require("node:path");
require("dotenv").config({
    path: path.resolve(__dirname, ".env")
});
// Importar módulos para la API, conexión DB, logs y formato
    JSON
const express = require('express');
const  mongoose  = require('mongoose');
const  morgan  = require('morgan');
const  bodyParser  = require('body-parser');

// Importar rutas de producto y categoria
const productoRoute = require('./routes/producto-router')
const categoriaRoute = require('./routes/categoria-router')


// Credenciales apuntando a la variable de entorno
const user = process.env.DB_USER;
const password = process.env.DB_PASSWORD;
const uri = `mongodb+srv://${user}:${password}@cluster0.
   exouyu8.mongodb.net/?appName=Cluster0`

const PORT = process.env.API_PORT || 3000;

const app = express() // Declara la variable app como método
    express()
// registro de códigos de estado 4xx y 5xx
app.use(morgan('dev'));
app.use(bodyParser.urlencoded({extended : true}))
app.use(bodyParser.json());
```

```
app.use('/api/producto', productoRoute);
app.use('/api/categoria', categoriaRoute);

app.listen(PORT, () => {
    console.log(`Conexión exitosa. Servidor en ejecución
       bajo el puerto: ${PORT}/`);
})

mongoose.connect(uri, { dbName: 'inventario'});
const database = mongoose.connection;
//
database.once('open', () => {
    console.log('Conexión exitosa al clúster DB')
    console.log(`API: http://localhost:${PORT}/api/`)
});
database.on('error', (err) => console.log(`Error de conexión
    de a la base de datos ${err}`));
```

3.3. Modelado de la Base de Datos NoSQL

Para interactuar con MongoDB Atlas, se utilizó **Mongoose**. Este Object Data Modeling (ODM) permitió definir reglas estrictas para datos cruciales (como el nombre y el precio), al mismo tiempo que habilitó el uso del tipo de dato `Mixed` para absorber las especificaciones técnicas variables (RAM, almacenamiento, tipo de conexión) sin romper el esquema.

Modelo de Mongoose para la colección de Productos (models/Producto.js)

```
// [INSERTE AQUI EL CODIGO DEL ESQUEMA DEL PRODUCTO (
   MONGOOSE)]
```

Modelo de Mongoose para la colección de Categoria (models/Categoria.js)

```
const mongoose = require("mongoose");
const { Schema } = mongoose;

const categoriaSchama = new Schema(
  {
    nombre: {
      type: String,
      required: true,
    },
    descripcion: {
      type: String,
      required: false,
    },
```

```
    codigo: {
      type: String,
      required: true,
      unique: true,
      match: /^CAT-\d{3,}$/,
    },
  },
  { timestamps: true },
);

const categoria = mongoose.model("categoria",
   categoriaSchama);
module.exports = categoria;
```

Modelo de Mongoose para la colección de Contador (models/Contador.js)

```
const mongoose = require ('mongoose');
const { Schema } = mongoose;

const contadorSchema = new Schema({
  _id: String,
  secuencia: { type: Number, default: 0 }
});

module.exports = mongoose.model('Contador', contadorSchema,
   'contador');
```

3.4. Controladores y Rutas

La lógica de negocio reside en los controladores, los cuales procesan la petición una vez que **body-parser** ha parseado el cuerpo del mensaje. Estos controladores ejecutan métodos asíncronos de **Mongoose** (`find`, `save`, `findByIdAndUpdate`) para retornar respuestas estandarizadas al cliente.

Controlador de Producto (controllers/producto-controller.js)

```
const mongoose = require("mongoose");
const Producto = require("../models/producto");

// Consulta método HTTP
// Consultar todos los Producto
const mostrarProductos = (req,res) => {
  Producto.find()
    .then((response) => res.json({ response }))
    .catch((error) =>
```

```
      res.status(500).json({ message: `Error al consultar
         productos: ${error}` }),
    );
};

// Consultar producto por el ID Body JSON
const verProductoIDBody = (req,res,next) => {
  const productoID = req.body.productoID;
  Producto.findById(productoID)
  .then(response => {
    if (!response) {
      return res.status(404).res.json({message: 'Producto no
          encontrado'});
    }
    res.json({response})
  })
  .catch(error => res.json({message: `ID del producto invá
     lido: ${error}` }));
}

// Consultar producto por el ID URI
const verProductoIDUri = (req, res) => {
  const productoID = req.params.id;
  if (!mongoose.isObjectIdOrHexString(productoID)) {
    return res.status(400).json({ message: "ID del producto
       inválido" });
  }

  Producto.findById(productoID)
    .then((response) => {
      if (!response) {
        return res.status(404).json({ message: "ID del
           producto inválido" });
      }
      return res.json({ response });
    })
    .catch((error) =>
      res.status(500).json({ message: `Error al consultar
         producto: ${error}` }),
    );
};

// Agregar producto
const agregarProducto = async (req, res) => {
  try {
```

```
    const { nombre, precio, stock, categoria_id, espec } =
       req.body;
    const nuevoProducto = new Producto({
      nombre,
      precio,
      stock,
      categoria_id,
      espec,
    });
    await nuevoProducto.save();
    res.status(201).json({
      mensaje: "Producto creado con éxito",
      datos: nuevoProducto,
    });
  } catch (error) {
    res.status(500).json({ message: `Error: ${error}` });
  }
};

// Actualizar producto
const actualizarProducto = (req, res, next) => {
  const productoID = req.body.productoID;
  if (!mongoose.isObjectIdOrHexString(productoID)) {
    return res.status(400).json({ message: "ID de producto
       invalido" });
  }

  const { nombre, precio, stock, categoria_id, espec } = req
     .body;
  const modProducto = {
    nombre,
    precio,
    stock,
    categoria_id,
    espec
  };

  Producto.findByIdAndUpdate(
    productoID,
    { $set: modProducto },
    { new: true, runValidators: true },
  )
    .then((response) => {
      if (!response) {
        return res.status(404).json({ message: "Producto no
           encontrado" });
```

```
      }
      return res.json({ response });
    })
    .catch((error) =>
      res.status(500).json({ message: `Error al actualizar
         producto: ${error}` }),
    );
};

// Eliminar producto
const eliminarProducto = (req, res, next) =>{
  const productoID = req.body.productoID;
  if (!mongoose.isObjectIdOrHexString(productoID)) {
    return res.status(400).json({ message: "ID de producto
       invalido" });
  }

  Producto.findByIdAndDelete(productoID)
  .then((response) => {
    if (!response) {
      return res.status(404).json({ message: "Producto no
         encontrado" });
    }
    return res.json({ response });
  })
  .catch((error) =>
    res.status(500).json({ message: `Error al eliminar
       producto: ${error}` }),
  );
}

module.exports = {
    agregarProducto,
    mostrarProductos,
    verProductoIDUri,
    verProductoIDBody,
    actualizarProducto,
    eliminarProducto
}
```

Rutas de Producto (routes/producto-router.js)

```
const express = require('express');
const router = express.Router();

const productoController = require('../controllers/producto-
```

```
   controller');

router.get('/',productoController.mostrarProductos);
router.post('/insert',productoController.agregarProducto);
router.post('/body',productoController.verProductoIDBody);
router.get('/uri/:id',productoController.verProductoIDUri);
router.put('/update',productoController.actualizarProducto);
router.delete('/delete',productoController.eliminarProducto)
   ;

module.exports = router;
```

Controlador de Categoría (controllers/categoria-controller.js)

```
const mongoose = require("mongoose");
const Categoria = require("../models/categoria");
const Contador = require("../models/contador");

const mostrarCategorias = (req, res) => {
  Categoria.find()
    .then((response) => res.json(response ))
    .catch((error) =>
      res.status(500).json({ message: `Error al consultar
         categorias: ${error}` }),
    );
};

//Consultar categoria por ID Body JSON
const verCategoriaIDBody = (req, res) => {
  const categoriaID = req.body.categoriaID;
  Categoria.findById(categoriaID)
  .then(response => {
    if (!response) {
      res.status(404).res.json({message: 'Categoría no
         encontrada'});
    }
    res.json({response})
  })
  .catch( error => res.json({message: `Error a consultar
     categoria: ${error}`}))
}

// Consultar categoria por ID URL
const verCategoriaIDUri = (req, res) => {
  const categoriaID = req.params.id;
  if (!mongoose.isObjectIdOrHexString(categoriaID)) {
```

```
    return res.status(400).json({ message: "ID de categoria
       invalido" });
  }

  Categoria.findById(categoriaID)
    .then((response) => {
      if (!response) {
        return res.status(404).json({ message: "Categoria no
            encontrada" });
      }
      return res.json({ response });
    })
    .catch((error) =>
      res.status(500).json({ message: `Error al consultar
         categoria: ${error}` }),
    );
};

// Agregar categoria
const agregarCategoria = async (req, res) => {
  try {
    const contador = await Contador.findOneAndUpdate(
      { _id: "categoria" },
      { $inc: { secuencia: 1 } },
      { new: true, upsert: true, setDefaultsOnInsert: true }
    );

    const codigo = `CAT-${String(contador.secuencia).
       padStart(3, "0")}`;
    const { nombre, descripcion } = req.body;
    const nuevaCategoria = new Categoria({
      nombre,
      descripcion,
      codigo
    });
    await nuevaCategoria.save();
    res.status(201).json({
      mensaje: "Categoria creada con éxito",
      datos: nuevaCategoria,
    });
  } catch (error) {
    res
      .status(500)
      .json({ message: `Error al agregar nueva categoria: ${
         error}` });
  }
```

```
};
// Actualizar categoria
const actualizarCategoria = (req, res, next) => {
  const { categoriaID, nombre, descripcion } = req.body;
  const modCategoria = {
    nombre,
    descripcion
  };
  Categoria.findByIdAndUpdate(categoriaID, { $set:
     modCategoria }, { new: true, runValidators: true })
    .then((response) => {
      if(!response){
        return res.status(404).json({message: 'categoria no
           encontrada'});
      }
      res.json({ response });

    })
    .catch((error) =>
      res.status(500).json({ message: `Error al actualizar
         categoria: ${error}` })
    );

};

// Eliminar categoria
const eliminarCategoria = (req, res, next) => {
  const categoriaID = req.body.categoriaID;
  if (!mongoose.isObjectIdOrHexString(categoriaID)) {
    return res.status(400).json({ message: "ID de categoria
       invalido" });
  }

  Categoria.findByIdAndDelete(categoriaID)
    .then((response) => {
      if (!response) {
        return res.status(404).json({ message: "Categoria no
           encontrada" });
      }
      return res.json({ response });
    })
    .catch((error) =>
      res.status(500).json({ message: `Error al eliminar
         categoria: ${error}` }),
    );
};
```

```
module.exports = {
  agregarCategoria,
  mostrarCategorias,
  verCategoriaIDUri,
  verCategoriaIDBody,
  actualizarCategoria,
  eliminarCategoria,
};
```

Rutas de Categoría (routes/categoria-router.js)

```
const express = require('express');
const router = express.Router();

const categoriaController = require('../controllers/
   categoria-controller');

router.get('/',categoriaController.mostrarCategorias);
router.post('/insert',categoriaController.agregarCategoria);
router.post('/body',categoriaController.verCategoriaIDBody);
router.get('/uri/:id',categoriaController.verCategoriaIDUri)
   ;
router.put('/update',categoriaController.actualizarCategoria
   );
router.delete('/delete',categoriaController.
   eliminarCategoria);

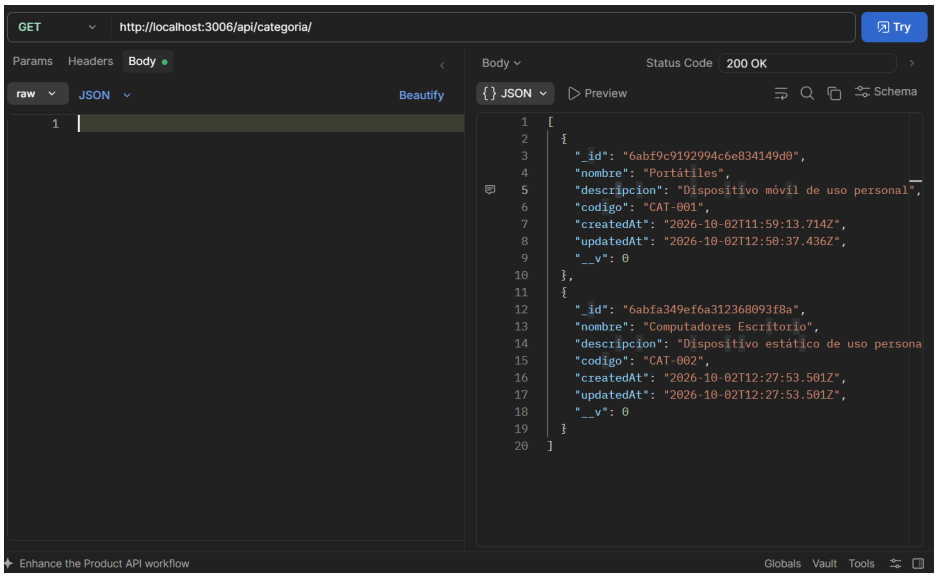
module.exports = router;
```

3.5. Pruebas y Evidencia de Ejecución (Comandos MongoDB Atlas)

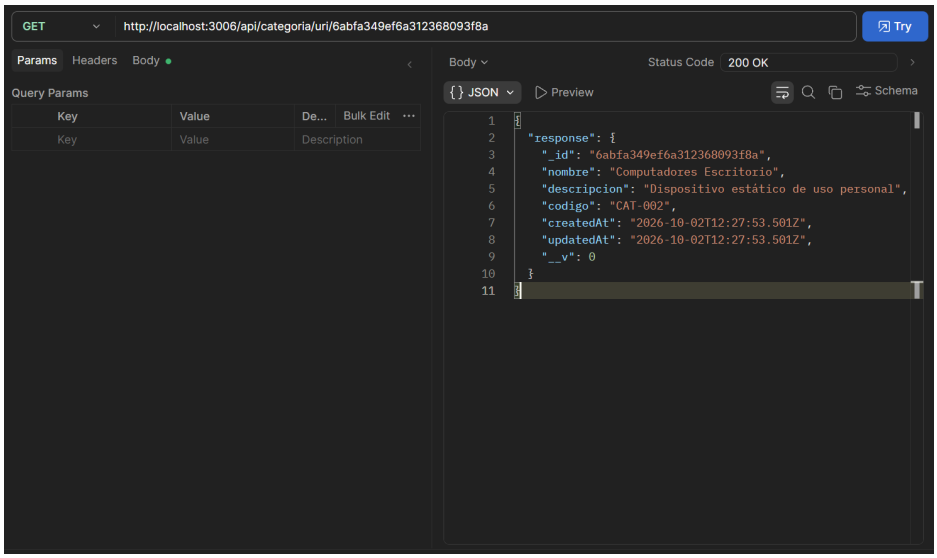
Para verificar la correcta recepción de datos y el funcionamiento íntegro de la base de datos, se ejecutaron pruebas de peticiones HTTP, las cuales fueron debidamente monitoreadas por **morgan** en la terminal y confirmadas en el cliente REST (Postman) y en el gestor de base de datos (MongoDB Compass/Atlas).

1. **Evidencias de las operaciones con categorías**

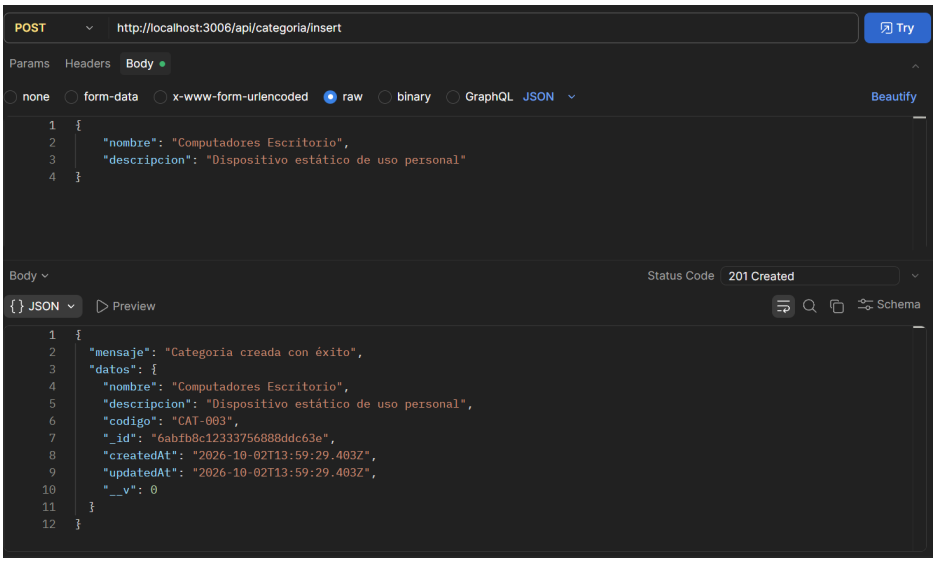
   - **GET: listar categorías.** Consulta todas las categorías registradas y muestra sus datos, como nombre, descripción y código.



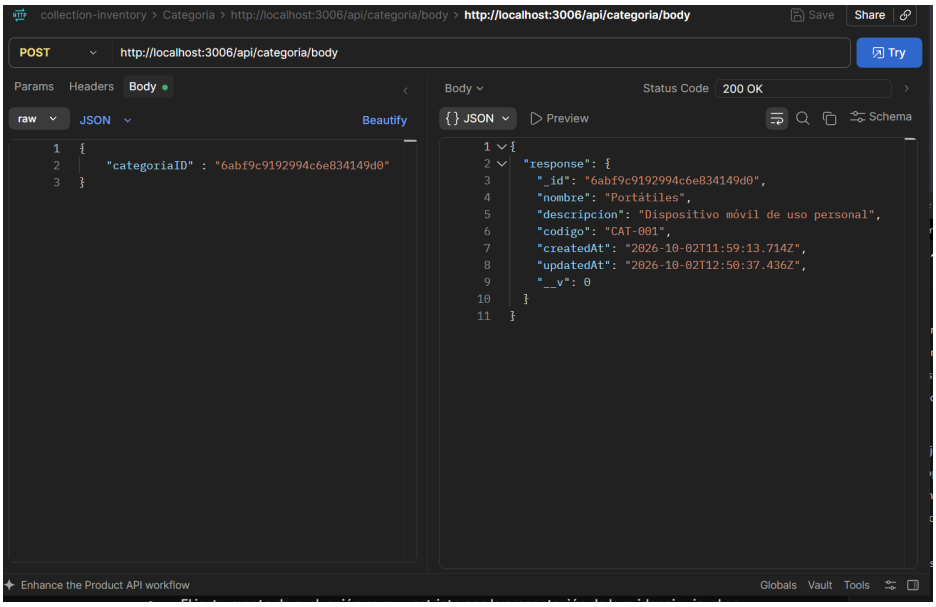
- **GET: consultar una categoría por URI.** Recupera una categoría específica a partir de su identificador incluido en la URL.



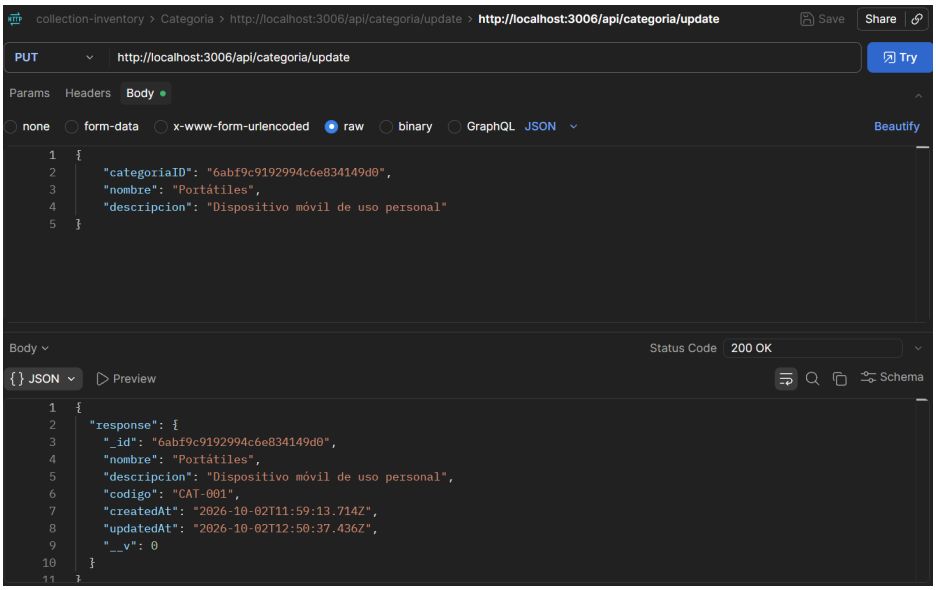
- **POST: crear una categoría.** Envía el nombre y la descripción en formato JSON; la respuesta confirma la creación y presenta el código asignado.



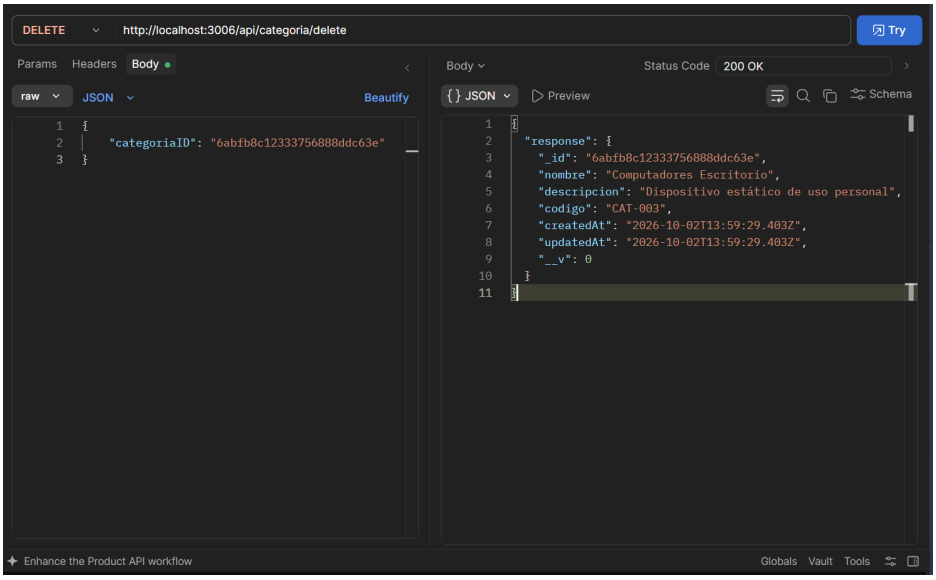
- **POST: consultar una categoría por ID en el cuerpo.** Envía `categoriaID` en el cuerpo JSON y recibe los datos de la categoría encontrada.



- **PUT: actualizar una categoría.** Envía el identificador y los campos editados; la respuesta muestra la categoría actualizada.

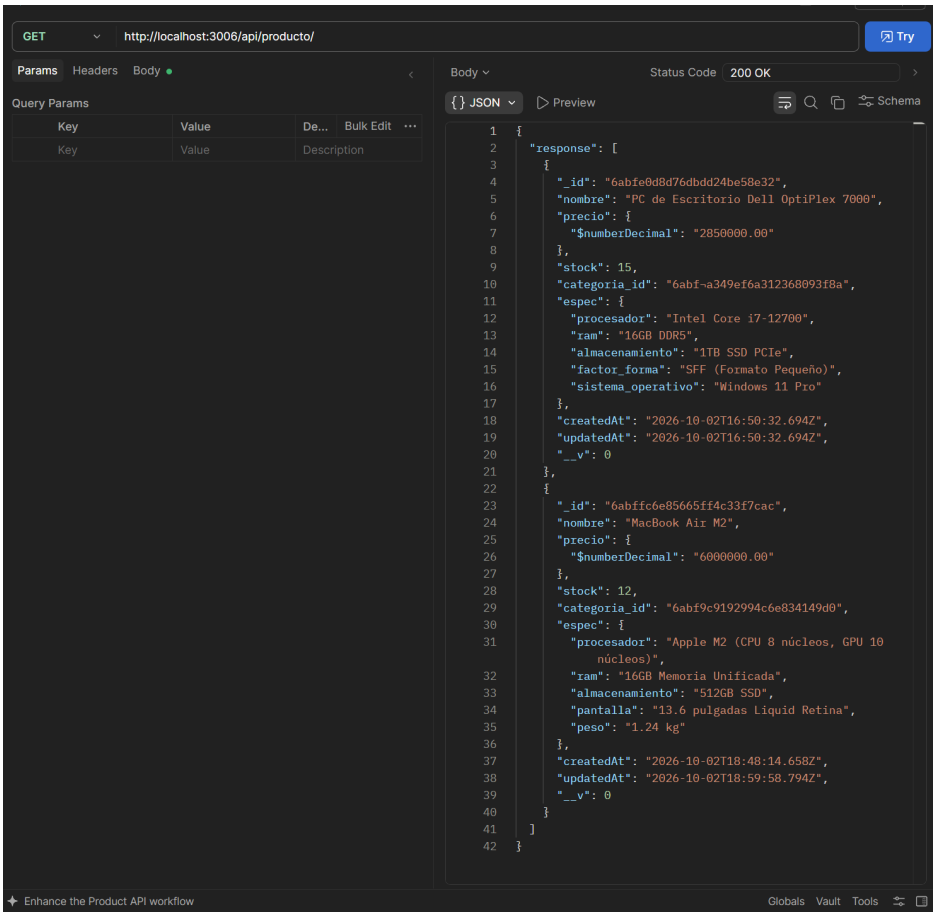


- **DELETE: eliminar una categoría.** Envía `categoriaID` en el cuerpo JSON y muestra el registro eliminado como confirmación.

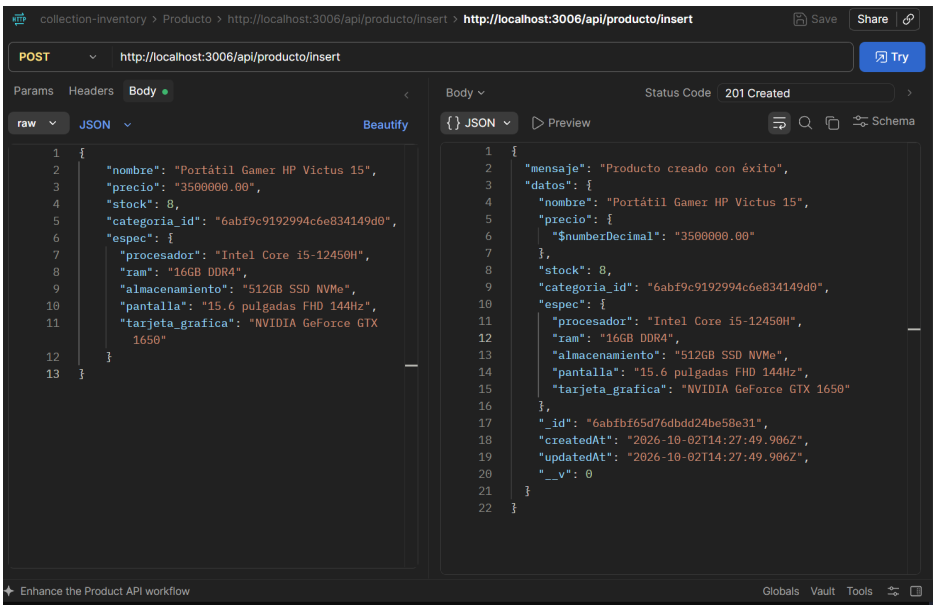


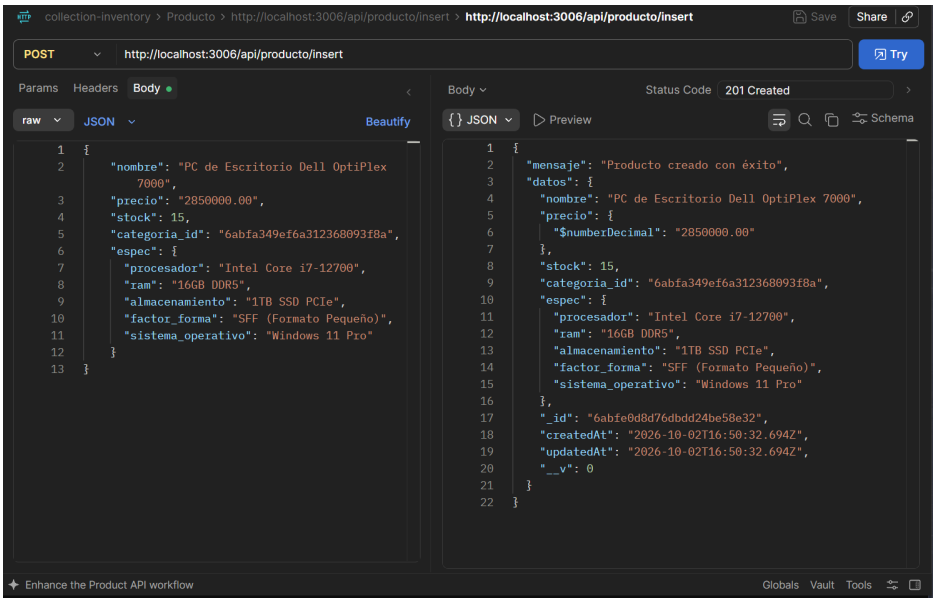
2. **Evidencias de las operaciones con productos**

   - **GET: listar productos.** Consulta los productos registrados, incluyendo su precio, existencias, categoría y especificaciones variables.

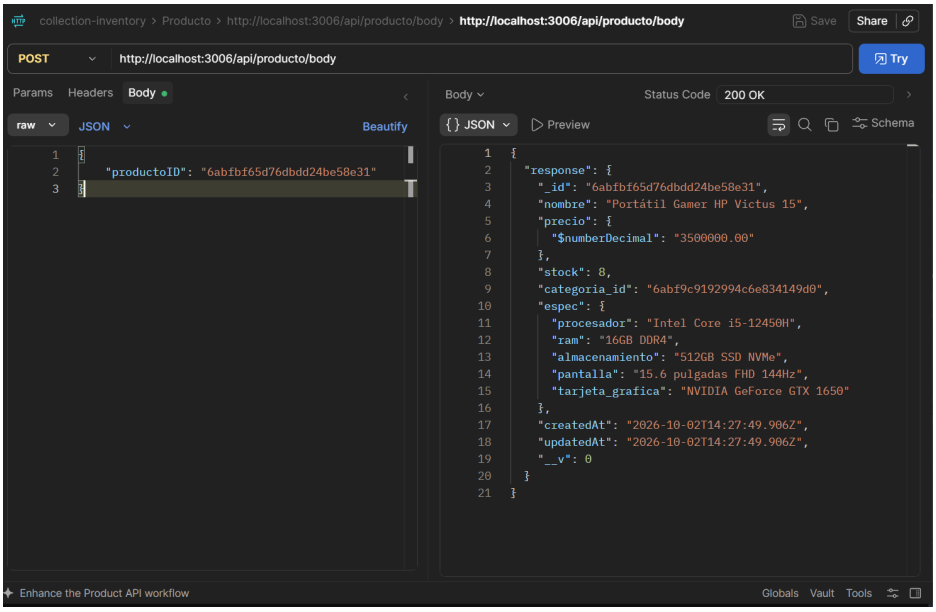


- **POST: crear productos.** Envía los datos del producto y sus especificaciones en formato JSON; las respuestas confirman la creación de los registros.

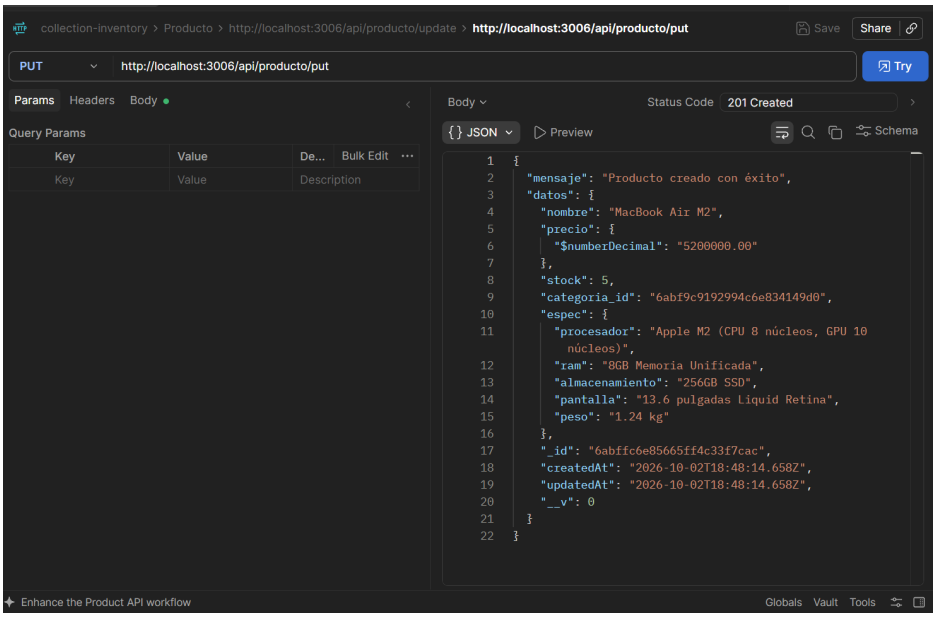


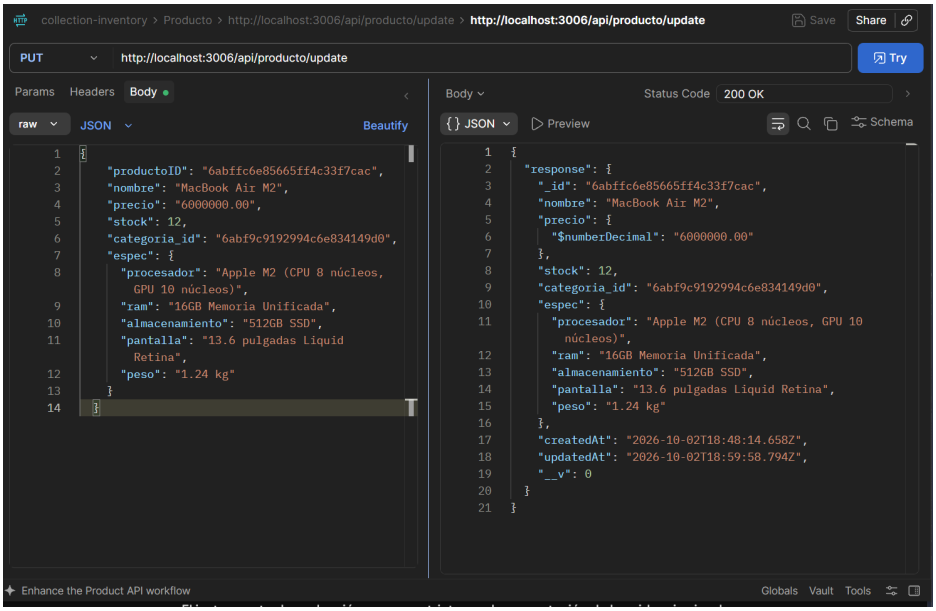


- **POST: consultar un producto por ID en el cuerpo.** Envía `productoID` en el cuerpo JSON y obtiene los datos del producto correspondiente.

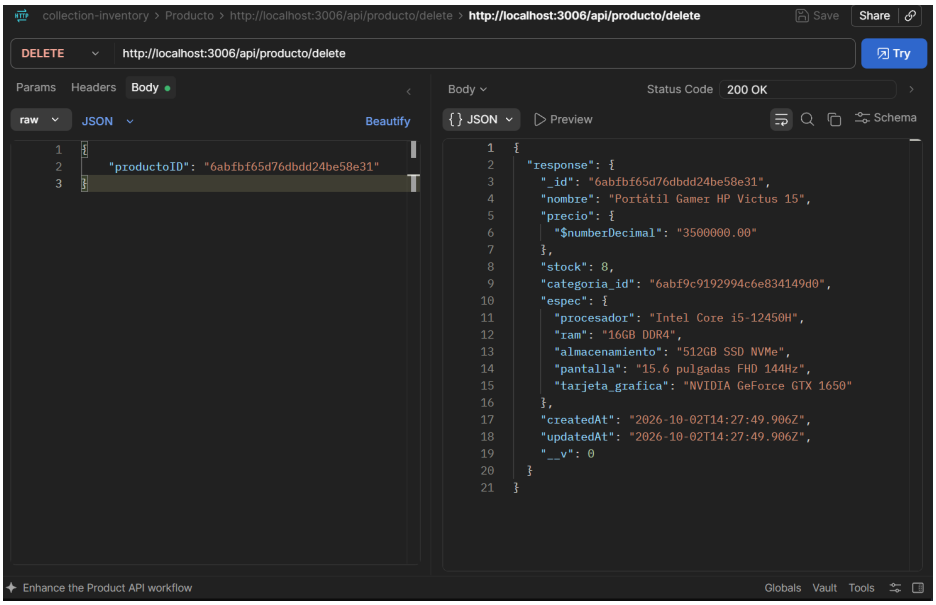


- **PUT: actualizar un producto.** La evidencia muestra la solicitud y respuesta antes y después de modificar sus campos, como precio y existencias.





- **DELETE: eliminar un producto.** Envía `productoID` en el cuerpo JSON y presenta el registro eliminado en la respuesta.


collection-inventory › Producto › http://localhost:3006/api/producto/delete › http://localhost:3006/api/producto/delete
Save
Share
DELETE
http://localhost:3006/api/producto/delete
Try
Params
Headers
Body
raw
JSON
Beautify
{
"productoID": "6abfbf65d76dbdd24be58e31"
}
Body
Status Code
200 OK
{ } JSON
Preview
Schema
{
"response": {
"_id": "6abfbf65d76dbdd24be58e31",
"nombre": "Portátil Gamer HP Victus 15",
"precio": {
"$numberDecimal": "3500000.00"
},
"stock": 8,
"categoria_id": "6abf9c9192994c6e834149d0",
"espec": {
"procesador": "Intel Core i5-12450H",
"ram": "16GB DDR4",
"almacenamiento": "512GB SSD NVMe",
"pantalla": "15.6 pulgadas FHD 144Hz",
"tarjeta_grafica": "NVIDIA GeForce GTX 1650"
},
"createdAt": "2026-10-02T14:27:49.906Z",
"updatedAt": "2026-10-02T14:27:49.906Z",
"__v": 0
}
}
Enhance the Product API workflow
Globals
Vault
Tools

4. CONCLUSIONES

- El uso del driver nativo de MongoDB en Node.js permite automatizar la migración masiva de datos estructurados en hojas de cálculo hacia colecciones NoSQL de manera eficiente y controlada.
- La flexibilidad de los esquemas documentales facilitó la estructuración de la información deportiva solicitada, permitiendo realizar operaciones de manipulación y filtrado directo sobre los documentos JSON cumpliendo con todos los criterios del instrumento de evaluación.